```
Started GET "/cable" for 127.0.0.1
ProductsChannel is transmitting the subscription confirmation
ProductsChannel is streaming from products
Started GET "/cable/" [WebSocket] for 127.0.0.1
Successfully upgraded to WebSocket
Turbo::StreamsChannel is transmitting the subscription confirmation
Turbo::StreamsChannel is streaming from products
```

Again, update the price of a book in one browser window and watch the catalog update instantly in any other browser window that shows the Depot store.

What We Just Did

In this iteration, we added Hotwire support to our cart:

- We moved the shopping cart into the sidebar. We then arranged for the `create` action to redisplay the catalog page.
- We used `as: :turbo_stream` to indicate to the `LineItemsController.create` that the client supports Turbo Streams.
- We then used an ERB partial template to return only the portions of the page that need to be replaced.
- We used Action Cable and Turbo Frames to update the catalog display whenever a product changes.
- We wrote a test that verifies not only the creation of a line item but also the content of the response that's returned from such a request.

The key point to take away is the incremental style of Hotwire development. Start with a conventional application and then add Ajax features, one by one. Hotwire applications can be hard to debug; by adding it slowly to an application, you make it easier to track down what changed if your application stops working. And, as you saw, starting with a conventional